LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Heiza Rizki Pratama

2509106019

A'25

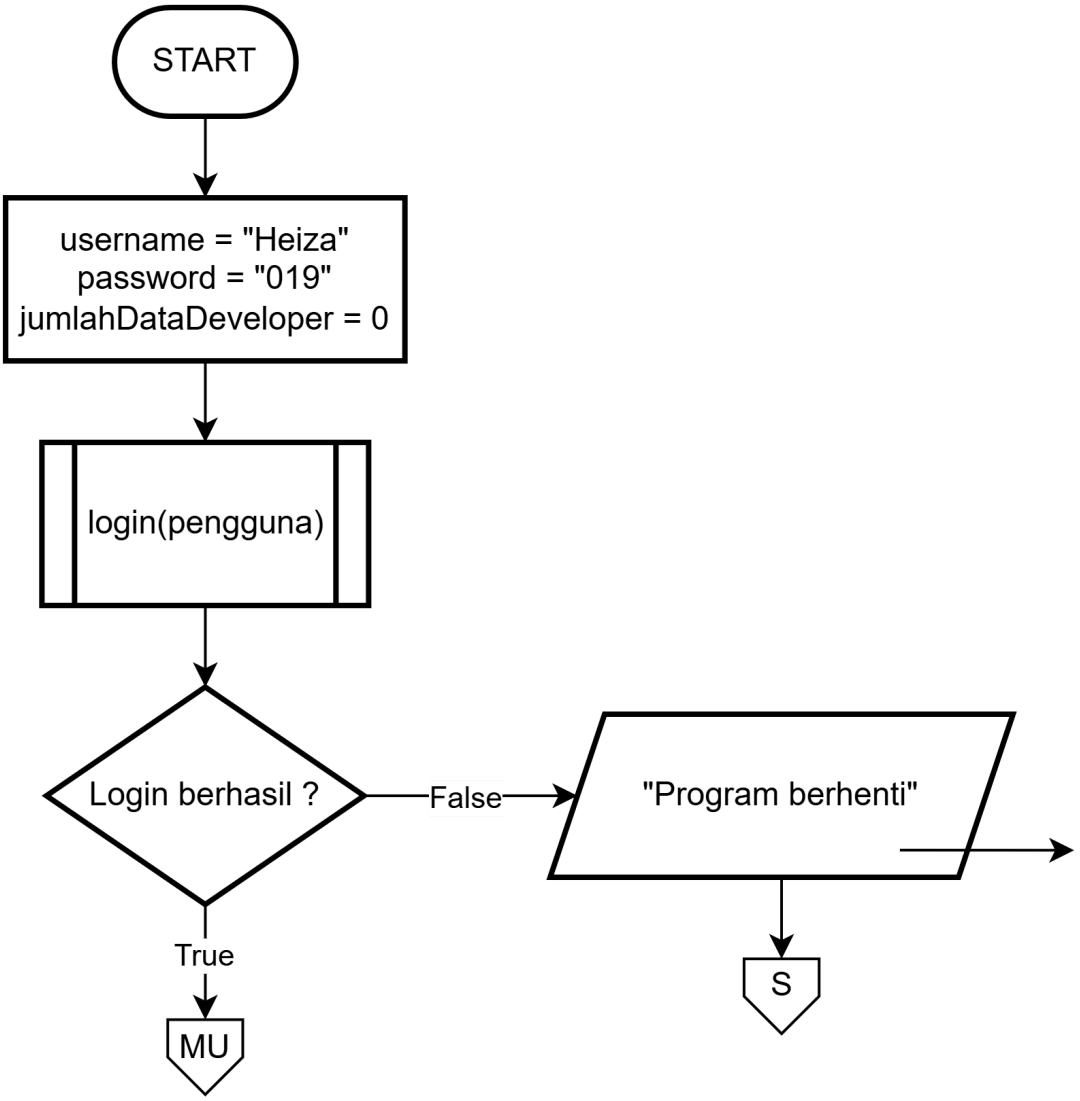
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

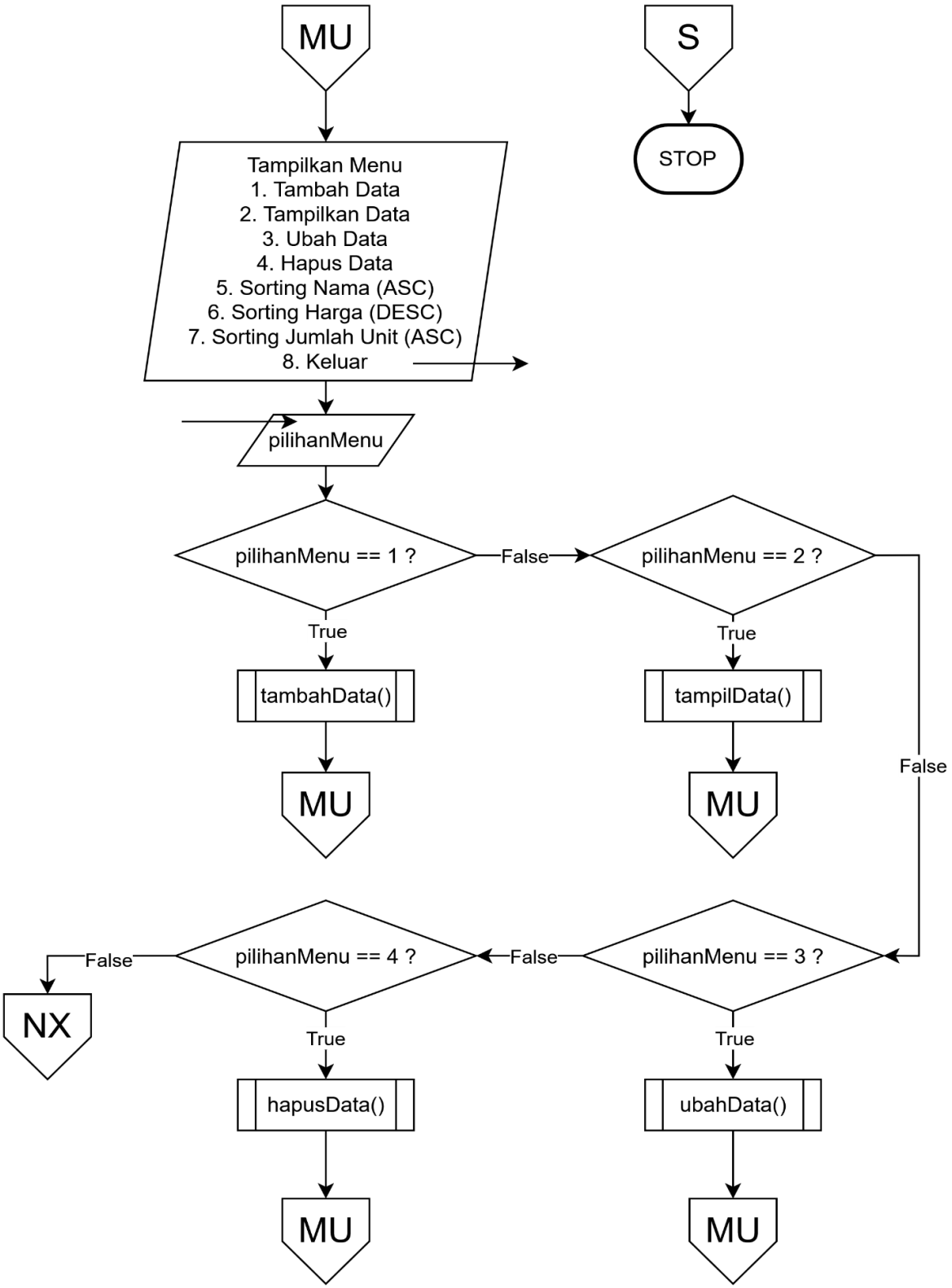
1. Flowchart



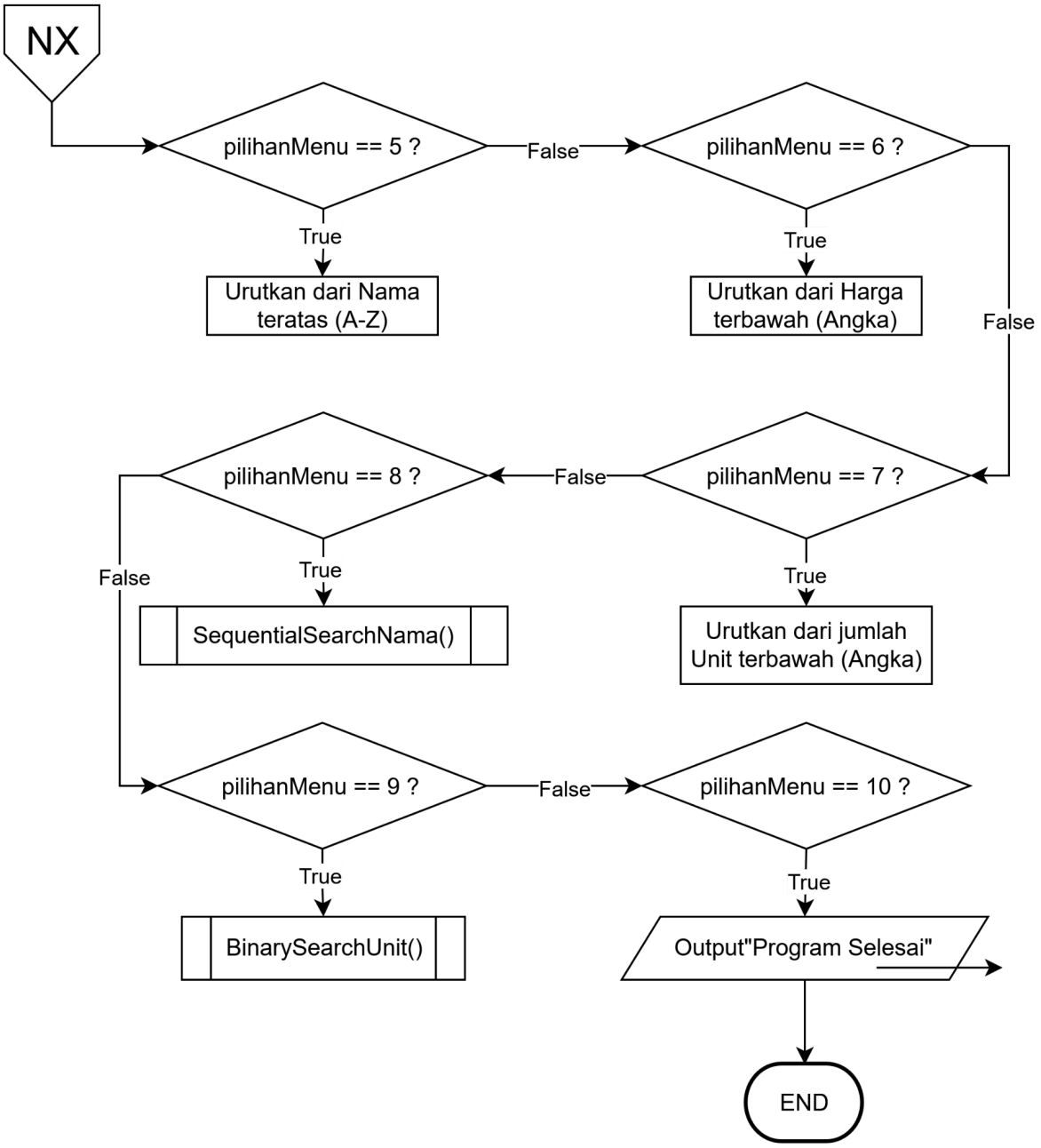
Gambar 1.1

Pada Flowchart diatas ini diperlihatkan proses awal program dan proses login pengguna sebelum dapat mengakses sistem.
Program dimulai dari simbol START. Setelah itu program melakukan inisialisasi beberapa variabel, yaitu menetapkan nilai awal username = "Heiza", password = "019", dan jumlahDataDeveloper = 0. Variabel ini digunakan sebagai data awal untuk proses autentikasi dan penyimpanan jumlah data developer.
Selanjutnya program memanggil subproses login(pengguna) yang berfungsi untuk menerima input username dan password dari pengguna. Proses ini menggunakan simbol predefined process, yang menandakan bahwa proses login didefinisikan dalam prosedur atau fungsi terpisah.

Setelah proses login dilakukan, program melakukan percabangan keputusan dengan kondisi “Login berhasil?”. Jika kondisi bernilai False (login gagal), maka program menampilkan pesan “Program berhenti” dan alur program menuju simbol STOP, yang berarti program tidak dapat dilanjutkan.
Namun jika kondisi bernilai True (login berhasil), maka alur program akan diarahkan ke connector MU, yang menghubungkan flowchart ini dengan flowchart berikutnya yaitu bagian menu utama program.
Dengan demikian, flowchart ini berfungsi sebagai mekanisme keamanan awal program untuk memastikan hanya pengguna yang memiliki username dan password yang benar yang dapat mengakses sistem.


MU
S
STOP
Tampilkan Menu
1. Tambah Data
2. Tampilkan Data
3. Ubah Data
4. Hapus Data
5. Sorting Nama (ASC)
6. Sorting Harga (DESC)
7. Sorting Jumlah Unit (ASC)
8. Keluar
pilihanMenu
pilihanMenu == 1 ?
False
pilihanMenu == 2 ?
True
True
tambahData()
tampilData()
MU
MU
False
pilihanMenu == 4 ?
False
pilihanMenu == 3 ?
False
NX
True
True
hapusData()
ubahData()
MU
MU

Gambar 1.2



Gambar 1.3

Menu Utama (Main Menu) merupakan bagian inti dari program yang berfungsi sebagai pusat kontrol seluruh fitur dalam sistem manajemen data developer perumahan. Pada bagian ini, pengguna diberikan berbagai pilihan menu yang dapat digunakan untuk melakukan operasi terhadap data, seperti menambah, menampilkan, mengubah, menghapus, melakukan pengurutan (sorting), serta
pencarian (searching).

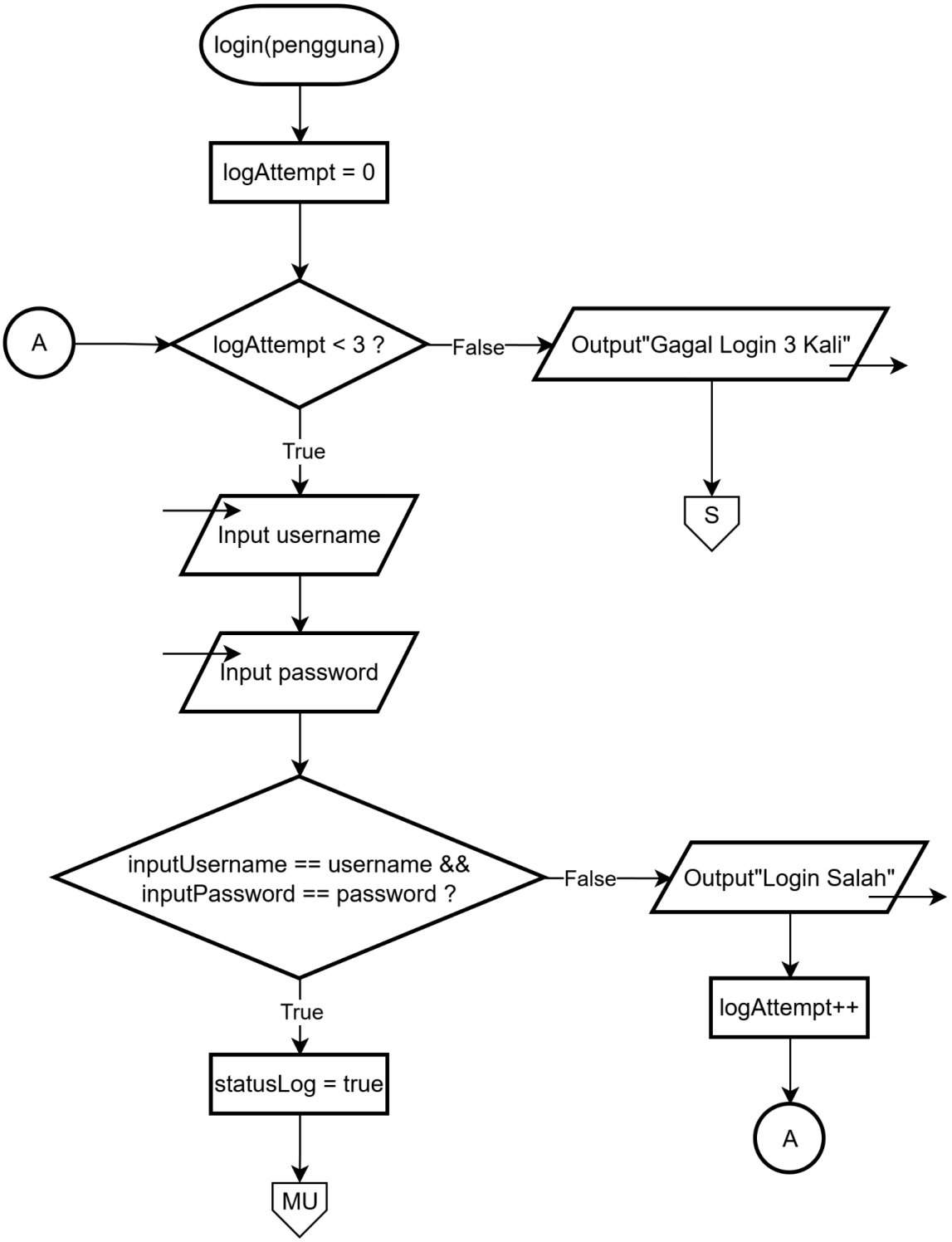
Alur program pada menu utama dimulai setelah pengguna berhasil melakukan proses login. Program kemudian menampilkan daftar menu yang berisi beberapa pilihan, yaitu:

1. Tambah Data
2. Tampilkan Data
3. Ubah Data
4. Hapus Data
5. Sorting Nama (Ascending)
6. Sorting Harga (Descending)
7. Sorting Jumlah Unit (Ascending)
8. Pencarian Nama Developer (Sequential Search)
9. Pencarian Jumlah Unit (Binary Search)
10. Keluar

Setelah menu ditampilkan, pengguna diminta untuk memasukkan pilihan sesuai dengan nomor menu yang tersedia. Input tersebut kemudian diproses menggunakan struktur percabangan (if-else) untuk menentukan aksi yang akan dijalankan.
Setiap pilihan menu memiliki fungsi masing-masing, yaitu:

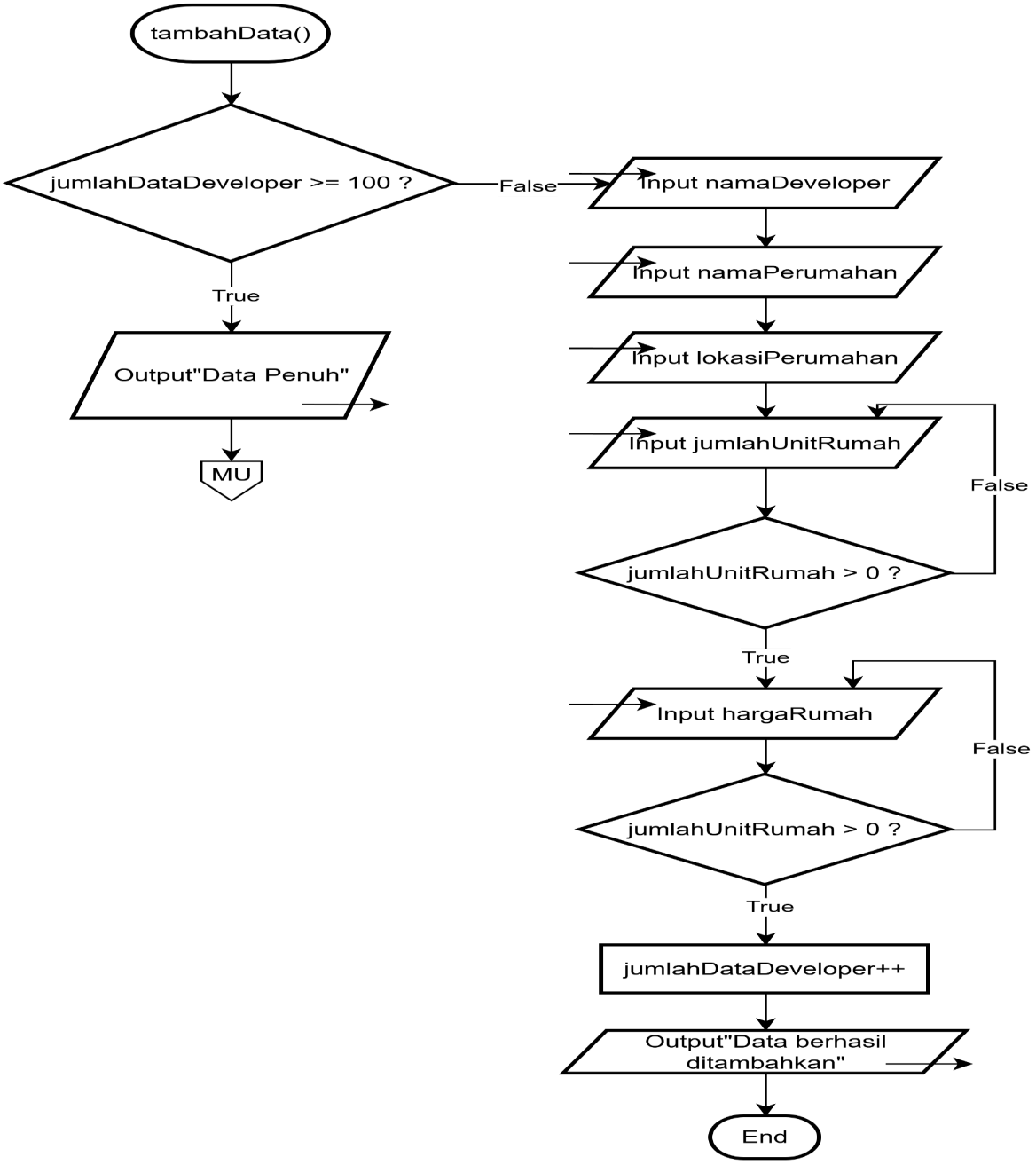
- ➔ Menu 1 (Tambah Data): Digunakan untuk menambahkan data developer baru ke dalam array. Data yang dimasukkan meliputi nama developer, nama perumahan, lokasi, jumlah unit, dan harga rumah.
- ➔ Menu 2 (Tampilkan Data): Menampilkan seluruh data yang telah tersimpan dalam bentuk tabel.
- ➔ Menu 3 (Ubah Data): Memungkinkan pengguna untuk mengubah data tertentu berdasarkan nomor indeks.
- ➔ Menu 4 (Hapus Data): Menghapus data berdasarkan nomor yang dipilih, kemudian menggeser data setelahnya.
- ➔ Menu 5–7 (Sorting): Digunakan untuk mengurutkan data berdasarkan kriteria tertentu, seperti nama, harga, dan jumlah unit.
- ➔ Menu 8–9 (Searching): Digunakan untuk mencari data berdasarkan nama developer atau jumlah unit rumah.
- ➔ Menu 10 (Keluar): Mengakhiri program.

Struktur menu utama menggunakan perulangan do-while, sehingga program akan terus berjalan dan kembali ke menu utama setelah suatu proses selesai dilakukan, selama pengguna belum memilih opsi keluar (menu 10). Hal ini memungkinkan pengguna untuk melakukan berbagai operasi secara berulang tanpa perlu menjalankan ulang program.
Dengan adanya menu utama ini, sistem menjadi lebih terstruktur dan interaktif, serta memudahkan pengguna dalam mengakses seluruh fitur yang tersedia.



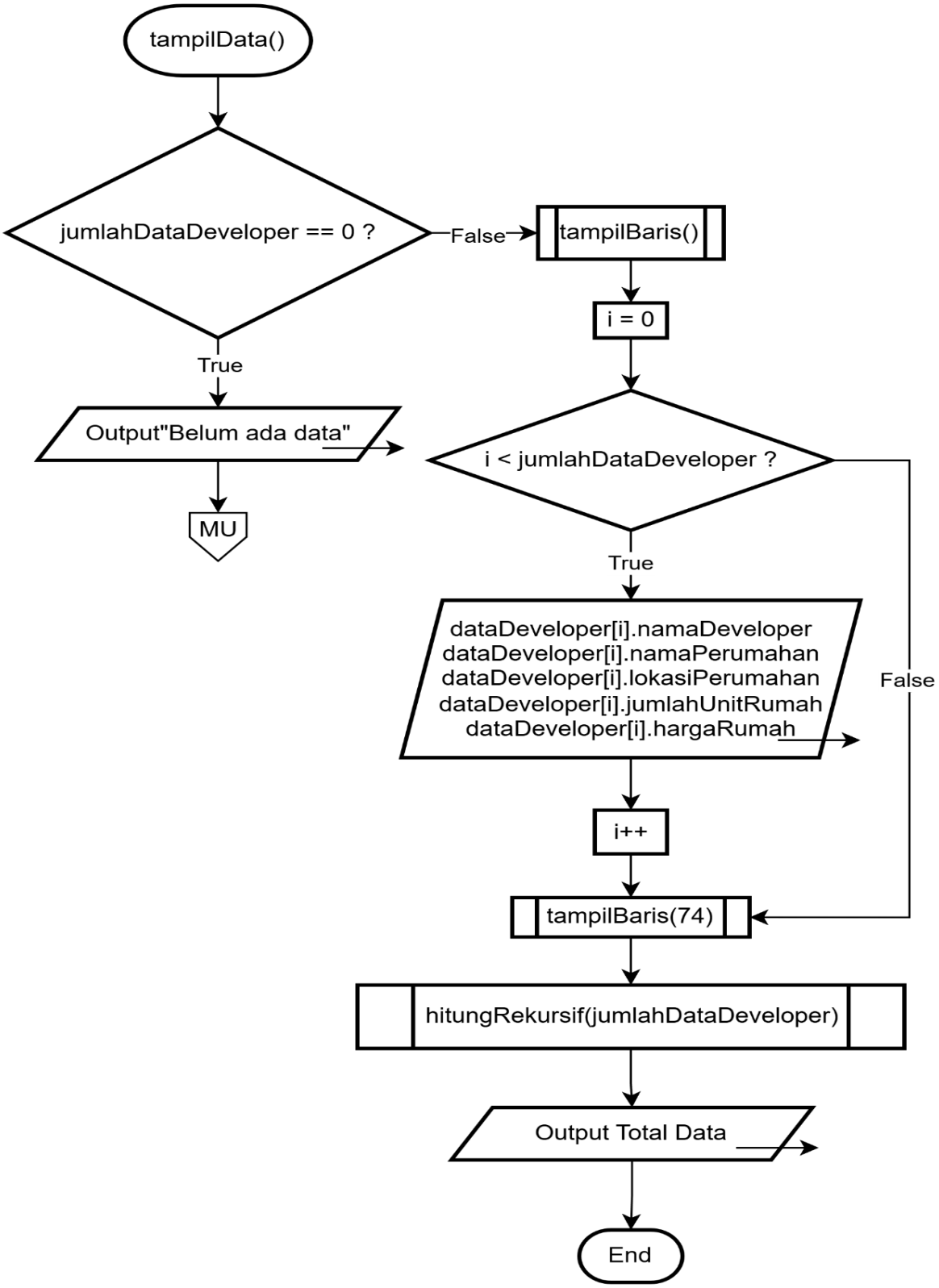
Gambar 1.3

Flowchart login menggambarkan proses autentikasi pengguna dengan batas maksimal 3 kali percobaan. Proses ini menggunakan perulangan dan percabangan untuk menentukan apakah pengguna berhasil masuk atau tidak.



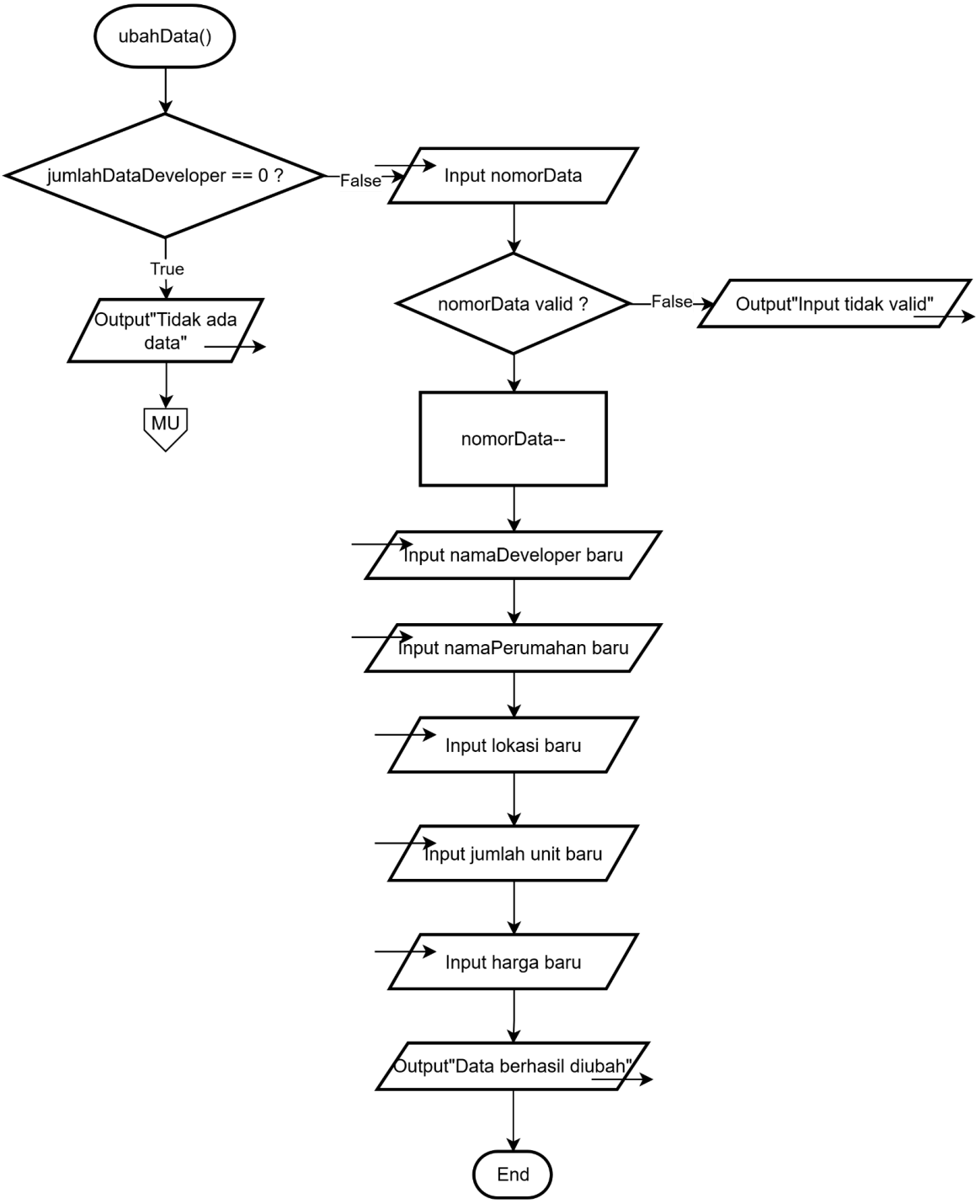
Gambar 1.4

Flowchart tambah data digunakan untuk menambahkan data perumahan ke dalam array dengan batas maksimal 100 data serta dilengkapi validasi input.



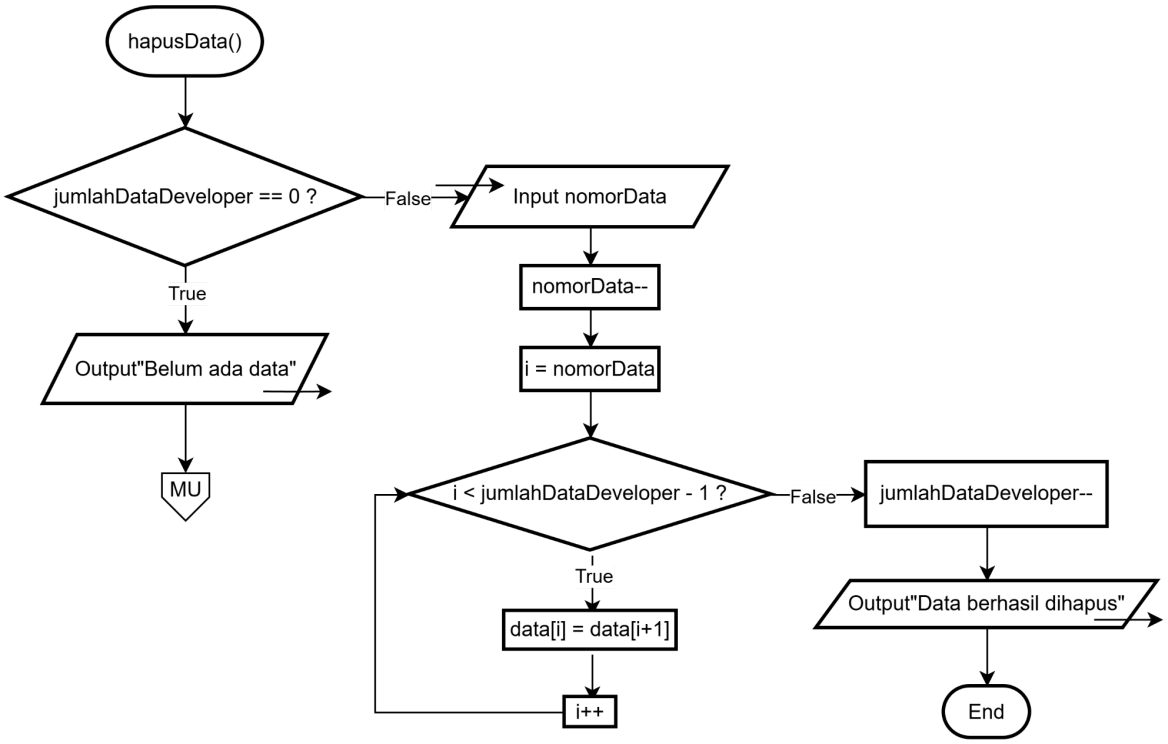
Gambar 1.5

Flowchart tampil data digunakan untuk menampilkan seluruh data developer/perumahan yang tersimpan, serta menampilkan total jumlah data.



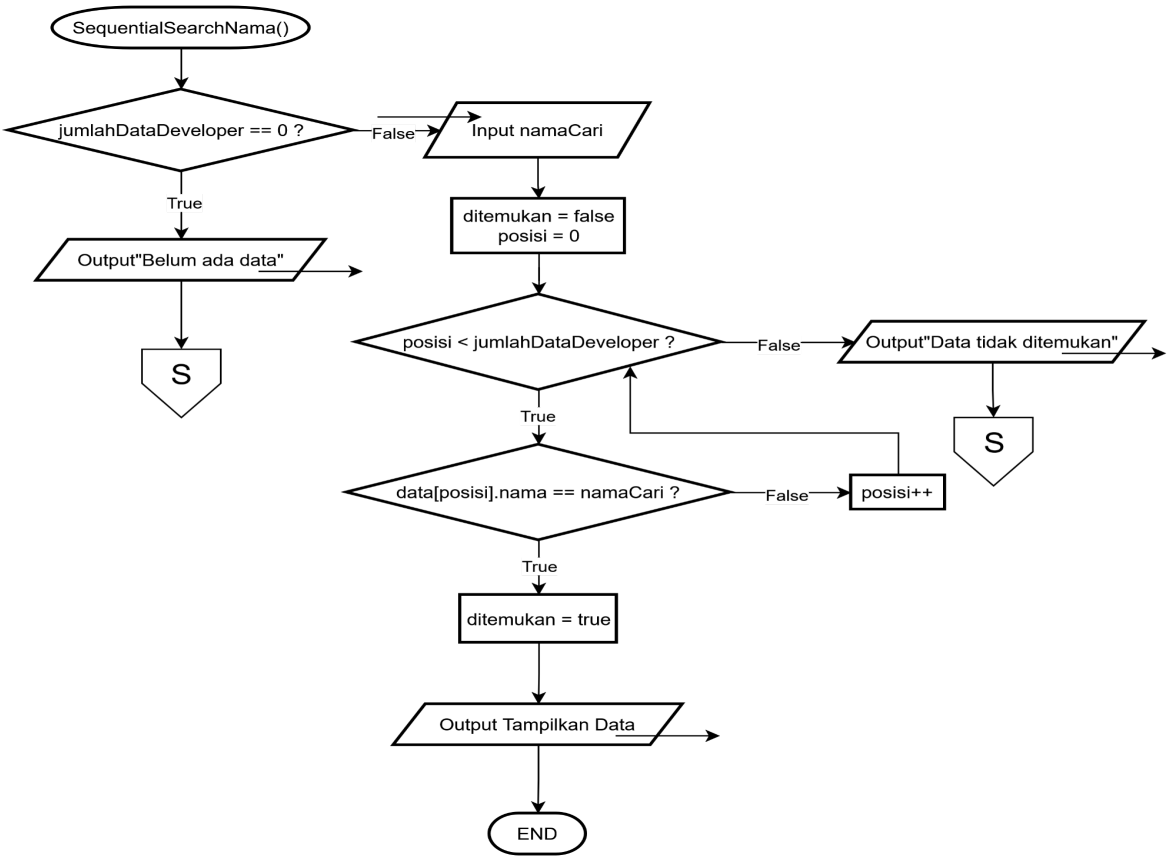
Gambar 1.6

Flowchart ubah data digunakan untuk mengubah data perumahan berdasarkan nomor data yang dipilih oleh user.



Gambar 1.7

Flowchart hapus data digunakan untuk menghapus data perumahan berdasarkan nomor yang dipilih, dengan cara menggeser elemen array dan mengurangi jumlah data.



Gambar 1.8

Sequential Search atau pencarian berurutan merupakan metode pencarian data yang dilakukan dengan cara memeriksa setiap elemen dalam array secara satu per satu, dimulai dari indeks pertama hingga elemen terakhir, sampai data yang dicari ditemukan atau seluruh data telah diperiksa.

Pada program ini, Sequential Search digunakan untuk mencari data berdasarkan nama developer.

Proses pencarian dimulai dengan pengecekan apakah data tersedia atau tidak. Jika jumlah data sama dengan nol, maka program akan langsung menampilkan pesan bahwa data belum tersedia, dan proses pencarian dihentikan.

Jika data tersedia, pengguna diminta untuk memasukkan nama developer yang ingin dicari. Setelah itu, program melakukan inisialisasi variabel, yaitu:

- ➔ ditemukan diset ke false sebagai penanda apakah data ditemukan atau tidak
- ➔ posisi diset ke 0 sebagai indeks awal pencarian

Selanjutnya, program masuk ke proses perulangan (loop) dengan kondisi selama posisi masih lebih kecil dari jumlah data. Pada setiap iterasi, program akan membandingkan nilai nama developer pada indeks saat ini dengan nama yang dicari.

Jika ditemukan kecocokan:

- ➔ Variabel ditemukan diubah menjadi true
- ➔ Data yang sesuai ditampilkan ke layar
- ➔ Proses pencarian dihentikan

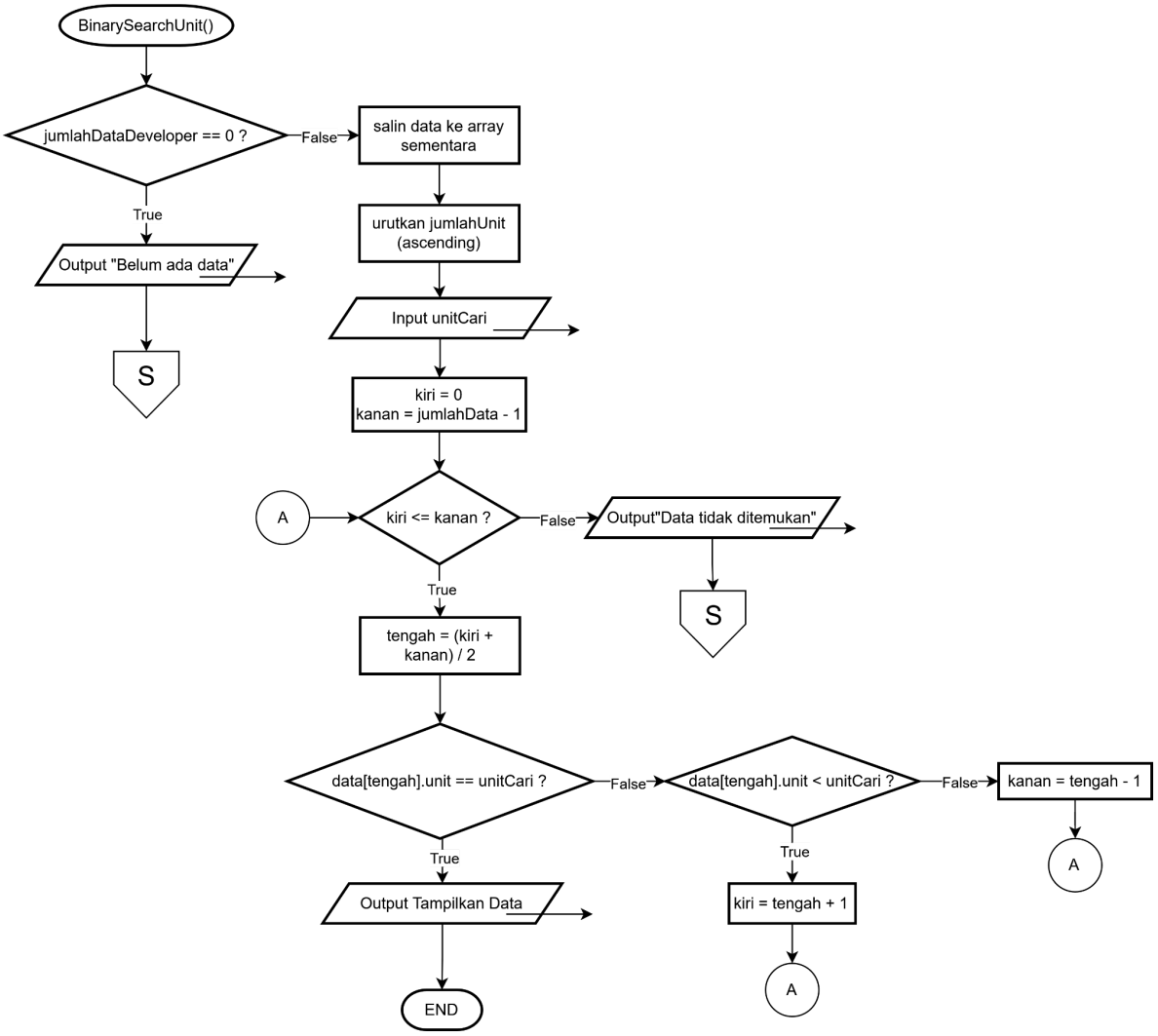
Jika tidak cocok:

- ➔ Nilai posisi akan ditambah satu

➔ Proses kembali ke iterasi berikutnya

Apabila seluruh data telah diperiksa dan tidak ditemukan kecocokan, maka program akan menampilkan pesan bahwa data tidak ditemukan.

Metode Sequential Search memiliki kelebihan yaitu mudah diimplementasikan dan tidak memerlukan data dalam kondisi terurut. Namun, kekurangannya adalah kurang efisien untuk jumlah data yang besar karena harus memeriksa satu per satu.



Gambar 1.9

Binary Search merupakan metode pencarian yang lebih efisien dibandingkan Sequential Search, karena menggunakan teknik pembagian data menjadi dua bagian secara berulang. Namun, metode ini memiliki syarat utama, yaitu data harus dalam keadaan terurut (sorted). Pada program ini, Binary Search digunakan untuk mencari data berdasarkan jumlah unit rumah.

Proses dimulai dengan pengecekan apakah data tersedia. Jika tidak ada data, maka program langsung menampilkan pesan bahwa data belum tersedia.

Jika data tersedia, langkah pertama yang dilakukan adalah menyalin data ke array sementara. Hal ini bertujuan agar proses pengurutan tidak mengubah data asli.

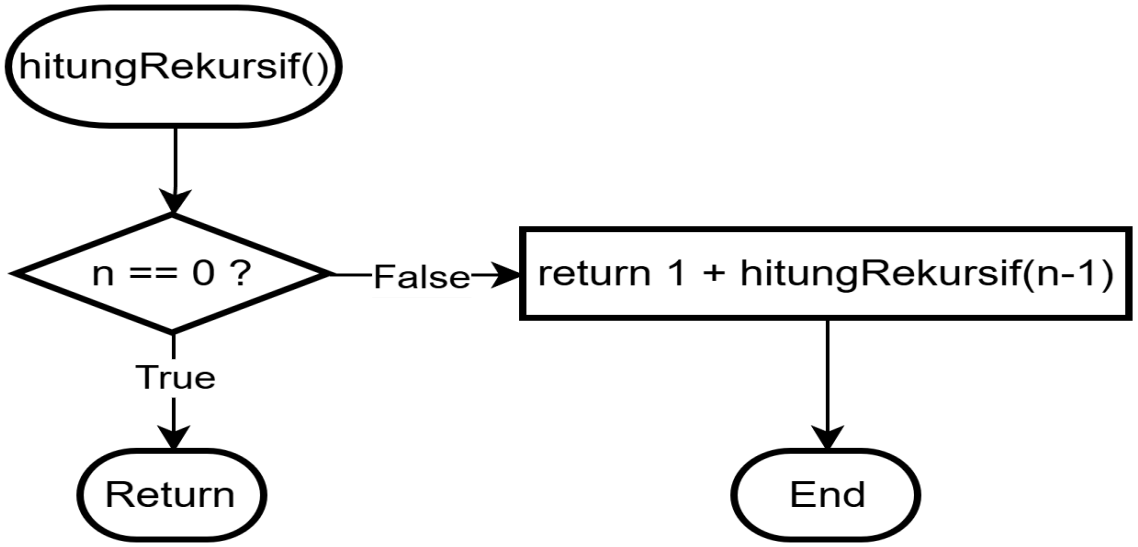
Selanjutnya, data dalam array sementara diurutkan berdasarkan jumlah unit rumah secara ascending menggunakan algoritma insertion sort. Proses pengurutan ini sangat penting karena Binary Search hanya dapat bekerja dengan benar pada data yang sudah terurut.
Setelah itu, pengguna diminta untuk memasukkan jumlah unit yang ingin dicari. Kemudian dilakukan inisialisasi:

- ➔ kiri sebagai indeks awal (0)
- ➔ kanan sebagai indeks akhir (jumlah data - 1)

Proses pencarian dilakukan dengan langkah berikut:

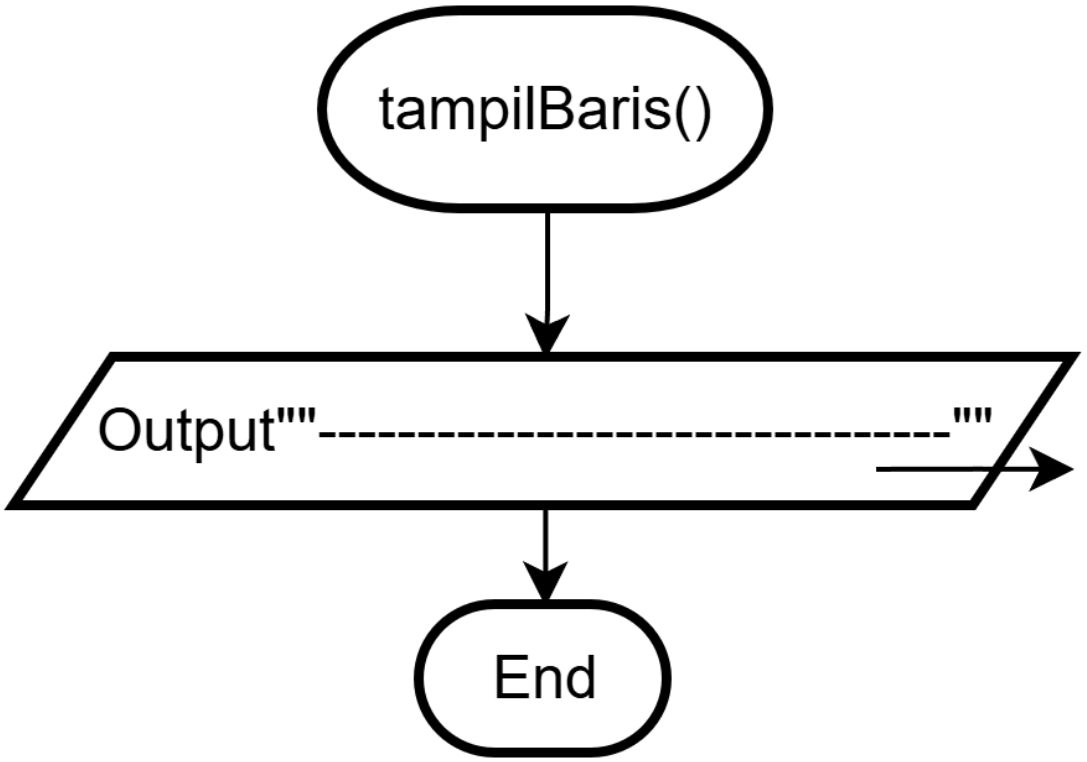
- ➔ Hitung nilai tengah dengan rumus:
  tengah = (kiri + kanan) / 2
- ➔ Bandingkan nilai pada posisi tengah dengan nilai yang dicari:
  - ◆ Jika sama, data ditemukan dan langsung ditampilkan
  - ◆ Jika nilai tengah lebih kecil dari target, maka pencarian dilanjutkan ke bagian kanan dengan menggeser nilai kiri
  - ◆ Jika nilai tengah lebih besar, maka pencarian dilanjutkan ke bagian kiri dengan menggeser nilai kanan
- ➔ Proses ini diulang selama nilai kiri masih kurang dari atau sama dengan nilai kanan.

Jika setelah perulangan selesai data tidak ditemukan, maka program akan menampilkan pesan bahwa data tidak ditemukan.
Binary Search memiliki kelebihan dalam hal efisiensi karena dapat mengurangi jumlah pencarian secara signifikan, dengan kompleksitas waktu O(log n). Namun, metode ini membutuhkan proses tambahan berupa pengurutan data sebelum pencarian dilakukan.



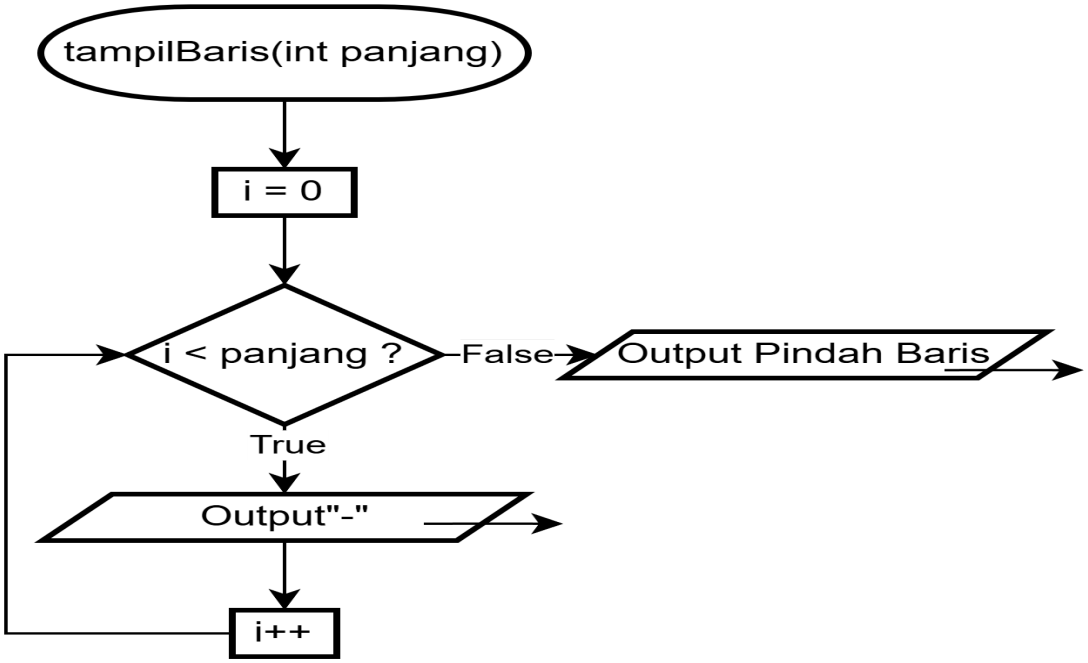
Gambar 2.0

Flowchart hitungRekursif() digunakan untuk menghitung jumlah data secara rekursif, yaitu dengan memanggil fungsi itu sendiri hingga mencapai kondisi dasar.



Gambar 2.1

Menampilkan garis pemisah panjang secara langsung.



Gambar 1.10

Menampilkan garis pemisah sesuai jumlah (panjang) dengan perulangan.

2. Deskripsi Singkat Program

Program ini merupakan Sistem Manajemen Data Developer Perumahan yang dibuat menggunakan bahasa pemrograman C++. Program diawali dengan proses login pengguna yang mengharuskan pengguna memasukkan username dan password yang telah ditentukan sebelumnya. Pengguna diberikan maksimal tiga kali percobaan login. Jika login gagal sebanyak tiga kali, maka program akan berhenti.

Setelah berhasil login, pengguna akan masuk ke menu utama yang menyediakan beberapa fitur untuk mengelola data perumahan yang dikembangkan oleh developer. Program ini menerapkan konsep CRUD (Create, Read, Update, Delete) pada data perumahan yang disimpan dalam bentuk array struktur.

Fitur-fitur yang tersedia dalam program ini meliputi:

1. Tambah Data Perumahan (Create) untuk menambahkan data developer dan perumahan baru.
2. Tampilkan Data Perumahan (Read) untuk menampilkan seluruh data perumahan yang telah disimpan.
3. Ubah Data Perumahan (Update) untuk memperbarui data perumahan yang sudah ada.
4. Hapus Data Perumahan (Delete) untuk menghapus data perumahan tertentu dari daftar.
5. Keluar (Exit) untuk mengakhiri program.

Program ini juga dilengkapi dengan validasi input untuk memastikan data yang dimasukkan pengguna sesuai dengan format yang diharapkan, seperti memastikan jumlah unit rumah dan harga rumah bernilai positif.

3. Source Code

```
#include <iostream>
#include <string>
#include <iomanip>
#include <conio.h>
#include <windows.h>
#include "validator.h"

using namespace std;

//KONSTANTA & WARNA KONSOL

const int KAPASITAS_MAKS = 100;

//Color Pallate Windows Console
enum KodeWarna {
    WARNA_PUTIH     = 7,
    WARNA_HIJAU     = 10,
    WARNA_MERAH     = 12,
```

```
    WARNA_KUNING   = 14,
    WARNA_BIRU_MUD = 11,
    WARNA_BIRU_TUA = 9,
    WARNA_UNGU     = 13
};

//STRUCT

struct Pengguna {
    string namaAkun;
    string kataSandi;
};

struct Developer {
    string namaDeveloper;
    string namaPerumahan;
    string lokasiPerumahan;
    int    jumlahUnitRumah;
    long   hargaRumah;
};

//FUNGSI UTILITAS TAMPILAN

void aturWarna(int kodeWarna) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), kodeWarna);
}

void resetWarna() {
    aturWarna(WARNA_PUTIH);
}

void cetakGaris(char karakter = '=', int panjang = 74) {
    for (int i = 0; i < panjang; i++) cout << karakter;
    cout << endl;
}

void cetakBanner() {
    system("cls");
    aturWarna(WARNA_BIRU_MUD);
    cetakGaris('=');
    cout << setw(45) << "  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN  " <<
endl;
    cetakGaris('=');
    resetWarna();
}

void cetakPesan(const string& pesan, int kodeWarna = WARNA_HIJAU) {
    aturWarna(kodeWarna);
```

```
    cout << endl << pesan << endl;
    resetWarna();
}

void cetakError(const string& pesanError) {
    aturWarna(WARNA_MERAH);
    cout << endl << pesanError << endl;
    resetWarna();
}

//FUNGSI REKURSIF  -  Menghitung jumlah data

int hitungRekursif(int jumlahDataDeveloper) {
    if (jumlahDataDeveloper == 0) return 0;
    return 1 + hitungRekursif(jumlahDataDeveloper - 1);
}

//ALGORITMA SORTING

void bubbleSortNamaAscending(Developer* dataDeveloper, int
jumlahDataDeveloper) {
    for (int i = 0; i < jumlahDataDeveloper - 1; i++) {
        for (int j = 0; j < jumlahDataDeveloper - i - 1; j++) {
            if (dataDeveloper[j].namaDeveloper > dataDeveloper[j +
1].namaDeveloper) {
                Developer temp         = dataDeveloper[j];
                dataDeveloper[j]       = dataDeveloper[j + 1];
                dataDeveloper[j + 1]   = temp;
            }
        }
    }
}

void selectionSortHargaDescending(Developer* dataDeveloper, int
jumlahDataDeveloper) {
    for (int i = 0; i < jumlahDataDeveloper - 1; i++) {
        int indeksMaks = i;
        for (int j = i + 1; j < jumlahDataDeveloper; j++) {
            if (dataDeveloper[j].hargaRumah >
dataDeveloper[indeksMaks].hargaRumah) {
                indeksMaks = j;
            }
        }
        Developer temp             = dataDeveloper[i];
        dataDeveloper[i]           = dataDeveloper[indeksMaks];
        dataDeveloper[indeksMaks]  = temp;
    }
}
```

```
void insertionSortUnitAscending(Developer* dataDeveloper, int
jumlahDataDeveloper) {
    for (int i = 1; i < jumlahDataDeveloper; i++) {
        Developer kunci = dataDeveloper[i];
        int j = i - 1;
        while (j >= 0 && dataDeveloper[j].jumlahUnitRumah >
kunci.jumlahUnitRumah) {
            dataDeveloper[j + 1] = dataDeveloper[j];
            j--;
        }
        dataDeveloper[j + 1] = kunci;
    }
}

//ALGORITMA PENCARIAN

void sequentialSearchNama(Developer* dataDeveloper, int jumlahDataDeveloper)
{
    cetakBanner();
    aturWarna(WARNA_KUNING);
    cout << "  [ CARI DATA - SEQUENTIAL SEARCH (NAMA DEVELOPER) ]" << endl;
    resetWarna();
    cetakGaris('-');

    try {
        if (jumlahDataDeveloper == 0) throw DataKosongException();

        string namaDicari;
        cout << "  Masukkan nama developer yang dicari : ";
        getline(cin, namaDicari);
        Validator::validasiStringTidakKosong(namaDicari, "Nama pencarian");

        bool  statusDitemukan  = false;
        int   indeksDitemukan  = -1;

        for (int posisi = 0; posisi < jumlahDataDeveloper; posisi++) {
            if (dataDeveloper[posisi].namaDeveloper == namaDicari) {
                statusDitemukan = true;
                indeksDitemukan = posisi;
                break;
            }
        }

        if (statusDitemukan) {
            cetakGaris('-');
            aturWarna(WARNA_HIJAU);
            cout << "  Data ditemukan pada nomor " << indeksDitemukan + 1 <<
```

```
endl;
            resetWarna();
            cetakGaris('-');
            cout << "  Nama Developer  : " <<
dataDeveloper[indeksDitemukan].namaDeveloper  << endl;
            cout << "  Nama Perumahan  : " <<
dataDeveloper[indeksDitemukan].namaPerumahan   << endl;
            cout << "  Lokasi          : " <<
dataDeveloper[indeksDitemukan].lokasiPerumahan << endl;
            cout << "  Jumlah Unit     : " <<
dataDeveloper[indeksDitemukan].jumlahUnitRumah << " unit" << endl;
            cout << "  Harga Rumah     : Rp " <<
dataDeveloper[indeksDitemukan].hargaRumah  << endl;
        } else {
            cetakPesan("  Data dengan nama \"" + namaDicari + "\" tidak
ditemukan.", WARNA_KUNING);
        }

    } catch (const DataKosongException& e) {
        cetakError(e.what());
    } catch (const InputTidakValidException& e) {
        cetakError(e.what());
    }

    cetakGaris('-');
    system("pause");
}

void binarySearchUnit(Developer* dataDeveloper, int jumlahDataDeveloper) {
    cetakBanner();
    aturWarna(WARNA_KUNING);
    cout << "  [ CARI DATA - BINARY SEARCH (JUMLAH UNIT RUMAH) ]" << endl;
    resetWarna();
    cetakGaris('-');

    try {
        if (jumlahDataDeveloper == 0) throw DataKosongException();

        // Salin data ke array temp, lalu sort ascending
        Developer dataSementara[KAPASITAS_MAKS];
        for (int i = 0; i < jumlahDataDeveloper; i++) {
            dataSementara[i] = dataDeveloper[i];
        }
        insertionSortUnitAscending(dataSementara, jumlahDataDeveloper);

        cout << "  Masukkan jumlah unit yang dicari : ";
        int unitDicari = Validator::bacaInteger("Jumlah unit");
        Validator::validasiPositif(unitDicari, "Jumlah unit");
```

```
        int  posisiKiri     = 0;
        int  posisiKanan    = jumlahDataDeveloper - 1;
        int  posisiTengah   = 0;
        bool statusDitemukan = false;
        int  indeksDitemukan = -1;

        while (posisiKiri <= posisiKanan) {
            posisiTengah = (posisiKiri + posisiKanan) / 2;
            if (dataSementara[posisiTengah].jumlahUnitRumah == unitDicari) {
                statusDitemukan = true;
                indeksDitemukan = posisiTengah;
                break;
            } else if (dataSementara[posisiTengah].jumlahUnitRumah <
unitDicari) {
                posisiKiri = posisiTengah + 1;
            } else {
                posisiKanan = posisiTengah - 1;
            }
        }

        if (statusDitemukan) {
            cetakGaris('-');
            aturWarna(WARNA_HIJAU);
            cout << "  Data ditemukan!" << endl;
            resetWarna();
            cetakGaris('-');
            cout << "  Nama Developer  : " <<
dataSementara[indeksDitemukan].namaDeveloper  << endl;
            cout << "  Nama Perumahan  : " <<
dataSementara[indeksDitemukan].namaPerumahan   << endl;
            cout << "  Lokasi          : " <<
dataSementara[indeksDitemukan].lokasiPerumahan << endl;
            cout << "  Jumlah Unit     : " <<
dataSementara[indeksDitemukan].jumlahUnitRumah << " unit" << endl;
            cout << "  Harga Rumah     : Rp " <<
dataSementara[indeksDitemukan].hargaRumah  << endl;
        } else {
            cetakPesan("  Data dengan jumlah unit " + to_string(unitDicari)
+ " tidak ditemukan.", WARNA_KUNING);
        }

    } catch (const DataKosongException& e) {
        cetakError(e.what());
    } catch (const InputTidakValidException& e) {
        cetakError(e.what());
    } catch (const NilaiNegatifException& e) {
        cetakError(e.what());
```

```
    }

    cetakGaris('-');
    system("pause");
}

//LOGIN

bool login(Pengguna* akun) {
    string inputNamaAkun;
    string inputKataSandi;
    int    jumlahPercobaan = 0;
    bool   statusBerhasil  = false;

    while (jumlahPercobaan < 3) {
        cetakBanner();
        aturWarna(WARNA_KUNING);
        cout << "  [ LOGIN SISTEM - PERCOBAAN " << jumlahPercobaan + 1 <<
"/3 ]" << endl;
        resetWarna();
        cetakGaris('-');

        cout << "  Username (Nama) : ";
        cin >> inputNamaAkun;

        cout << "  Password (NIM)  : ";
        inputKataSandi = "";
        char karakterInput;

        while ((karakterInput = _getch()) != 13) {
            if (karakterInput == 8) {
                if (!inputKataSandi.empty()) {
                    cout << "\b \b";
                    inputKataSandi.pop_back();
                }
            } else {
                cout << "*";
                inputKataSandi += karakterInput;
            }
        }
        cout << endl;

        if (inputNamaAkun == akun->namaAkun && inputKataSandi ==
akun->kataSandi) {
            statusBerhasil = true;
            break;
        } else {
            cetakError("  Login salah! Periksa kembali username dan password
```

```
Anda.");
            jumlahPercobaan++;
            system("pause");
        }
    }

    try {
        if (!statusBerhasil) throw LoginGagalException();
    } catch (const LoginGagalException& e) {
        cetakError(e.what());
        system("pause");
        return false;
    }

    aturWarna(WARNA_HIJAU);
    cout << endl << "  Login berhasil! Selamat datang, " << akun->namaAkun
<< "!" << endl;
    resetWarna();
    system("pause");
    return true;
}

//OPERASI CRUD

void tambahData(Developer* dataDeveloper, int* jumlahDataDeveloper) {
    cetakBanner();
    aturWarna(WARNA_KUNING);
    cout << "  [ TAMBAH DATA DEVELOPER PERUMAHAN ]" << endl;
    resetWarna();
    cetakGaris('-');

    try {
        if (*jumlahDataDeveloper >= KAPASITAS_MAKS) throw
DataPenuhException();

        string inputNama, inputPerumahan, inputLokasi;
        int    inputUnit;
        long   inputHarga;

        cout << "  Nama Developer   : ";
        getline(cin, inputNama);
        Validator::validasiStringTidakKosong(inputNama, "Nama Developer");

        cout << "  Nama Perumahan   : ";
        getline(cin, inputPerumahan);
        Validator::validasiStringTidakKosong(inputPerumahan, "Nama
Perumahan");
```

```
        cout << "  Lokasi Perumahan : ";
        getline(cin, inputLokasi);
        Validator::validasiStringTidakKosong(inputLokasi, "Lokasi");

        cout << "  Jumlah Unit Rumah: ";
        inputUnit = Validator::bacaInteger("Jumlah Unit");
        Validator::validasiPositif(inputUnit, "Jumlah Unit Rumah");

        cout << "  Harga Rumah (Rp) : ";
        inputHarga = Validator::bacaLong("Harga Rumah");
        Validator::validasiPositifLong(inputHarga, "Harga Rumah");

        //Simpan ke array
        dataDeveloper[*jumlahDataDeveloper].namaDeveloper   = inputNama;
        dataDeveloper[*jumlahDataDeveloper].namaPerumahan    =
inputPerumahan;
        dataDeveloper[*jumlahDataDeveloper].lokasiPerumahan  = inputLokasi;
        dataDeveloper[*jumlahDataDeveloper].jumlahUnitRumah  = inputUnit;
        dataDeveloper[*jumlahDataDeveloper].hargaRumah       = inputHarga;
        (*jumlahDataDeveloper)++;

        cetakPesan("  [OK] Data berhasil ditambahkan!", WARNA_HIJAU);

    } catch (const DataPenuhException& e) {
        cetakError(e.what());
    } catch (const InputTidakValidException& e) {
        cetakError(e.what());
    } catch (const NilaiNegatifException& e) {
        cetakError(e.what());
    }

    cetakGaris('-');
    system("pause");
}

void tampilData(Developer* dataDeveloper, int jumlahDataDeveloper) {
    cetakBanner();
    aturWarna(WARNA_KUNING);
    cout << "  [ DAFTAR DATA DEVELOPER PERUMAHAN ]" << endl;
    resetWarna();
    cetakGaris('-');

    try {
        if (jumlahDataDeveloper == 0) throw DataKosongException();

        // Header tabel
        aturWarna(WARNA_BIRU_MUD);
        cout << left
```

```
            << setw(4)  << "No"
            << setw(18) << "Nama Developer"
            << setw(20) << "Nama Perumahan"
            << setw(15) << "Lokasi"
            << setw(8)  << "Unit"
            << setw(16) << "Harga (Rp)"
            << endl;
        resetWarna();
        cetakGaris('-');

        for (int i = 0; i < jumlahDataDeveloper; i++) {
            cout << left
                << setw(4)  << i + 1
                << setw(18) << dataDeveloper[i].namaDeveloper
                << setw(20) << dataDeveloper[i].namaPerumahan
                << setw(15) << dataDeveloper[i].lokasiPerumahan
                << setw(8)  << dataDeveloper[i].jumlahUnitRumah
                << setw(16) << dataDeveloper[i].hargaRumah
                << endl;
        }

        cetakGaris('-');
        aturWarna(WARNA_KUNING);
        cout << "  Total data (rekursif) : " <<
hitungRekursif(jumlahDataDeveloper) << " entri" << endl;
        resetWarna();

    } catch (const DataKosongException& e) {
        cetakError(e.what());
    }

    cetakGaris('-');
    system("pause");
}

void ubahData(Developer* dataDeveloper, int jumlahDataDeveloper) {
    cetakBanner();
    aturWarna(WARNA_KUNING);
    cout << "  [ UBAH DATA DEVELOPER ]" << endl;
    resetWarna();
    cetakGaris('-');

    try {
        if (jumlahDataDeveloper == 0) throw DataKosongException();

        cout << "  Masukkan nomor data (1-" << jumlahDataDeveloper << ") :
";
        int nomorData = Validator::bacaInteger("Nomor data");
```

```
        Validator::validasiRentang(nomorData, 1, jumlahDataDeveloper, "Nomor
data");

        int indeksData = nomorData - 1;

        string inputNama, inputPerumahan, inputLokasi;
        int    inputUnit;
        long   inputHarga;

        cout << "  Nama Developer Baru  : ";
        getline(cin, inputNama);
        Validator::validasiStringTidakKosong(inputNama, "Nama Developer");

        cout << "  Nama Perumahan Baru  : ";
        getline(cin, inputPerumahan);
        Validator::validasiStringTidakKosong(inputPerumahan, "Nama
Perumahan");

        cout << "  Lokasi Baru          : ";
        getline(cin, inputLokasi);
        Validator::validasiStringTidakKosong(inputLokasi, "Lokasi");

        cout << "  Jumlah Unit Baru     : ";
        inputUnit = Validator::bacaInteger("Jumlah Unit");
        Validator::validasiPositif(inputUnit, "Jumlah Unit Rumah");

        cout << "  Harga Baru (Rp)      : ";
        inputHarga = Validator::bacaLong("Harga Rumah");
        Validator::validasiPositifLong(inputHarga, "Harga Rumah");

        dataDeveloper[indeksData].namaDeveloper   = inputNama;
        dataDeveloper[indeksData].namaPerumahan    = inputPerumahan;
        dataDeveloper[indeksData].lokasiPerumahan  = inputLokasi;
        dataDeveloper[indeksData].jumlahUnitRumah  = inputUnit;
        dataDeveloper[indeksData].hargaRumah       = inputHarga;

        cetakPesan("  [OK] Data berhasil diubah!", WARNA_HIJAU);

    } catch (const DataKosongException& e) {
        cetakError(e.what());
    } catch (const InputTidakValidException& e) {
        cetakError(e.what());
    } catch (const NilaiNegatifException& e) {
        cetakError(e.what());
    }

    cetakGaris('-');
    system("pause");
```

```
}

void hapusData(Developer* dataDeveloper, int* jumlahDataDeveloper) {
    cetakBanner();
    aturWarna(WARNA_KUNING);
    cout << "  [ HAPUS DATA DEVELOPER ]" << endl;
    resetWarna();
    cetakGaris('-');

    try {
        if (*jumlahDataDeveloper == 0) throw DataKosongException();

        cout << "  Masukkan nomor data (1-" << *jumlahDataDeveloper << ") :
";
        int nomorData = Validator::bacaInteger("Nomor data");
        Validator::validasiRentang(nomorData, 1, *jumlahDataDeveloper,
"Nomor data");

        int indeksData = nomorData - 1;

        //Geser data ke kiri
        for (int i = indeksData; i < (*jumlahDataDeveloper) - 1; i++) {
            dataDeveloper[i] = dataDeveloper[i + 1];
        }
        (*jumlahDataDeveloper)--;

        cetakPesan("  [OK] Data nomor " + to_string(nomorData) + " berhasil
dihapus!", WARNA_HIJAU);

    } catch (const DataKosongException& e) {
        cetakError(e.what());
    } catch (const InputTidakValidException& e) {
        cetakError(e.what());
    }

    cetakGaris('-');
    system("pause");
}

//FUNGSI TAMPIL MENU UTAMA

void tampilMenuUtama() {
    cetakBanner();
    aturWarna(WARNA_BIRU_MUD);
    cout << "  MENU UTAMA" << endl;
    resetWarna();
    cetakGaris('-');
    cout << "  [1]  Tambah Data" << endl;
```

```
    cout << "  [2]  Tampilkan Data" << endl;
    cout << "  [3]  Ubah Data" << endl;
    cout << "  [4]  Hapus Data" << endl;
    cetakGaris('-');
    aturWarna(WARNA_KUNING);
    cout << "  -- SORTING --" << endl;
    resetWarna();
    cout << "  [5]  Sort Nama Developer (Bubble Sort  - Ascending)" << endl;
    cout << "  [6]  Sort Harga Rumah    (Selection Sort - Descending)" <<
endl;
    cout << "  [7]  Sort Jumlah Unit    (Insertion Sort - Ascending)" <<
endl;
    cetakGaris('-');
    aturWarna(WARNA_KUNING);
    cout << "  -- PENCARIAN --" << endl;
    resetWarna();
    cout << "  [8]  Cari Nama Developer  (Sequential Search)" << endl;
    cout << "  [9]  Cari Jumlah Unit     (Binary Search)" << endl;
    cetakGaris('-');
    aturWarna(WARNA_MERAH);
    cout << "  [10] Keluar" << endl;
    resetWarna();
    cetakGaris('=');
    cout << "  Pilih Menu : ";
}

//MAIN

int main() {
    Pengguna akun;
    akun.namaAkun  = "Heiza";
    akun.kataSandi = "019";

    if (!login(&akun)) return 0;

    Developer dataDeveloper[KAPASITAS_MAKS];
    int       jumlahDataDeveloper = 0;
    int       pilihanMenu         = 0;

    do {
        tampilMenuUtama();

        try {
            pilihanMenu = Validator::bacaInteger("Pilihan menu");
            Validator::validasiRentang(pilihanMenu, 1, 10, "Pilihan menu");

        } catch (const InputTidakValidException& e) {
            cetakError(e.what());
```

```
            system("pause");
            continue;
        }

        switch (pilihanMenu) {
            case 1:
                tambahData(dataDeveloper, &jumlahDataDeveloper);
                break;
            case 2:
                tampilData(dataDeveloper, jumlahDataDeveloper);
                break;
            case 3:
                ubahData(dataDeveloper, jumlahDataDeveloper);
                break;
            case 4:
                hapusData(dataDeveloper, &jumlahDataDeveloper);
                break;
            case 5:
                try {
                    if (jumlahDataDeveloper == 0) throw
DataKosongException();
                    bubbleSortNamaAscending(dataDeveloper,
jumlahDataDeveloper);
                    cetakPesan("  [OK] Data berhasil diurutkan berdasarkan
Nama (Ascending).", WARNA_HIJAU);
                } catch (const DataKosongException& e) {
                    cetakError(e.what());
                }
                system("pause");
                break;
            case 6:
                try {
                    if (jumlahDataDeveloper == 0) throw
DataKosongException();
                    selectionSortHargaDescending(dataDeveloper,
jumlahDataDeveloper);
                    cetakPesan("  [OK] Data berhasil diurutkan berdasarkan
Harga (Descending).", WARNA_HIJAU);
                } catch (const DataKosongException& e) {
                    cetakError(e.what());
                }
                system("pause");
                break;
            case 7:
                try {
                    if (jumlahDataDeveloper == 0) throw
DataKosongException();
                    insertionSortUnitAscending(dataDeveloper,
```

```
jumlahDataDeveloper);
                    cetakPesan("  [OK] Data berhasil diurutkan berdasarkan
Jumlah Unit (Ascending).", WARNA_HIJAU);
                } catch (const DataKosongException& e) {
                    cetakError(e.what());
                }
                system("pause");
                break;
            case 8:
                sequentialSearchNama(dataDeveloper, jumlahDataDeveloper);
                break;
            case 9:
                binarySearchUnit(dataDeveloper, jumlahDataDeveloper);
                break;
            case 10:
                cetakBanner();
                aturWarna(WARNA_HIJAU);
                cout << "  Terima kasih telah menggunakan sistem ini. Sampai
jumpa!" << endl;
                resetWarna();
                cetakGaris('=');
                break;
            default:
                break;
        }

    } while (pilihanMenu != 10);

    return 0;
}
```

Source Code External

```
#ifndef VALIDATOR_H
#define VALIDATOR_H
#include <iostream>
#include <string>
#include <stdexcept>
#include <limits>

class DataPenuhException : public std::exception {
public:
    const char* what() const noexcept override {
        return "[!] ERROR: Data sudah penuh! Kapasitas maksimal 100 entri.";
    }
```

```
};

class DataKosongException : public std::exception {
public:
    const char* what() const noexcept override {
        return "[!] ERROR: Belum ada data yang tersimpan.";
    }
};

class IndexTidakValidException : public std::exception {
public:
    const char* what() const noexcept override {
        return "[!] ERROR: Nomor data yang dimasukkan tidak valid.";
    }
};

class LoginGagalException : public std::exception {
public:
    const char* what() const noexcept override {
        return "[!] AKSES DITOLAK: Anda telah gagal login 3 kali
berturut-turut.";
    }
};

class InputTidakValidException : public std::exception {
private:
    std::string pesanKesalahan;
public:
    explicit InputTidakValidException(const std::string& pesan)
        : pesanKesalahan("[!] INPUT TIDAK VALID: " + pesan) {}

    const char* what() const noexcept override {
        return pesanKesalahan.c_str();
    }
};

class NilaiNegatifException : public std::exception {
private:
    std::string pesanKesalahan;
public:
    explicit NilaiNegatifException(const std::string& namaField)
        : pesanKesalahan("[!] ERROR: " + namaField + " tidak boleh bernilai
nol atau negatif.") {}

    const char* what() const noexcept override {
        return pesanKesalahan.c_str();
    }
};
```

```
namespace Validator {

    // Membaca integer dari cin; lempar exception jika gagal
    int bacaInteger(const std::string& namaField) {
        int nilai;
        if (!(std::cin >> nilai)) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            throw InputTidakValidException(namaField + " harus berupa
bilangan bulat.");
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        return nilai;
    }

    // Membaca long dari cin; throw exception jika gagal
    long bacaLong(const std::string& namaField) {
        long nilai;
        if (!(std::cin >> nilai)) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            throw InputTidakValidException(namaField + " harus berupa
angka.");
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        return nilai;
    }

    // Validasi integer dalam rentang tertentu
    int validasiRentang(int nilai, int nilaiMin, int nilaiMax,
                        const std::string& namaField) {
        if (nilai < nilaiMin || nilai > nilaiMax) {
            throw InputTidakValidException(
                namaField + " harus antara " +
                std::to_string(nilaiMin) + " dan " +
                std::to_string(nilaiMax) + "."
            );
        }
        return nilai;
    }

    // Validasi string tidak kosong
    std::string validasiStringTidakKosong(const std::string& teks,
                                          const std::string& namaField) {
```

```
        if (teks.empty()) {
            throw InputTidakValidException(namaField + " tidak boleh
kosong.");
        }
        return teks;
    }

    // Validasi nilai positif (int)
    int validasiPositif(int nilai, const std::string& namaField) {
        if (nilai <= 0) {
            throw NilaiNegatifException(namaField);
        }
        return nilai;
    }

    // Validasi nilai positif (long)
    long validasiPositifLong(long nilai, const std::string& namaField) {
        if (nilai <= 0) {
            throw NilaiNegatifException(namaField);
        }
        return nilai;
    }

}

#endif
```

4. PENJELASAN PROGRAM

Program ini merupakan aplikasi berbasis bahasa C++ yang digunakan untuk mengelola data developer perumahan. Sistem ini dirancang untuk mempermudah pengguna dalam melakukan pengolahan data seperti menambah, menampilkan, mengubah, menghapus, mengurutkan, serta mencari data developer.
Data developer disimpan dalam sebuah array dengan kapasitas maksimal 100 data. Setiap data memiliki beberapa atribut, yaitu:

- Nama Developer
- Nama Perumahan
- Lokasi Perumahan
- Jumlah Unit Rumah
- Harga Rumah

Selain itu, program juga dilengkapi dengan fitur keamanan berupa login, sehingga hanya pengguna yang memiliki username dan password yang dapat mengakses sistem.

4.2 Struktur Data

Program menggunakan struktur (struct) untuk merepresentasikan data developer sebagai berikut:

```
struct Developer {
  string namaDeveloper;
  string namaPerumahan;
  string lokasiPerumahan;
  int jumlahUnitRumah;
  long hargaRumah;
};
```

Struktur ini digunakan untuk menyimpan seluruh informasi terkait developer dalam satu kesatuan data.

4.3 Penjelasan Fungsi-Fungsi Program

4.3.1 Fungsi tampilBaris()

Fungsi ini digunakan untuk menampilkan garis pembatas pada tampilan program agar output lebih rapi dan mudah dibaca.
Terdapat dua versi fungsi:

1. Tanpa parameter → menampilkan garis dengan panjang tetap
2. Dengan parameter → menampilkan garis sesuai panjang yang diinginkan

Fungsi ini tidak mengembalikan nilai dan hanya berfungsi untuk memperindah tampilan.

4.3.2 Fungsi hitungRekursif()

Fungsi ini digunakan untuk menghitung jumlah data menggunakan metode rekursif.
Cara kerja fungsi:

- Jika jumlah data = 0, maka fungsi mengembalikan 0
- Jika tidak, fungsi akan memanggil dirinya sendiri hingga mencapai kondisi dasar

Fungsi ini digunakan untuk menampilkan total data pada menu tampilan data.

4.3.3 Fungsi Sorting

Program menyediakan tiga metode pengurutan data, yaitu:

a. Bubble Sort (Ascending berdasarkan Nama)

Fungsi:

bubbleSortNamaAscending()

Digunakan untuk mengurutkan data berdasarkan nama developer dari A sampai Z.
Metode ini bekerja dengan cara membandingkan dua data yang berdekatan, kemudian menukarnya jika urutannya salah.

---

b. Selection Sort (Descending berdasarkan Harga)

Fungsi:
selectionSortHargaDescending()
Digunakan untuk mengurutkan data berdasarkan harga rumah dari yang tertinggi ke terendah.
Algoritma ini mencari nilai terbesar, kemudian menempatkannya di posisi yang sesuai.

---

c. Insertion Sort (Ascending berdasarkan Jumlah Unit)

Fungsi:
insertionSortUnitAscending()
Digunakan untuk mengurutkan data berdasarkan jumlah unit rumah dari kecil ke besar.
Metode ini juga digunakan sebagai persiapan sebelum melakukan binary search.

4.3.4 Fungsi Sequential Search

Fungsi:
sequentialSearchNama()
Digunakan untuk mencari data developer berdasarkan nama.
Cara kerja:

1. Pengguna memasukkan nama yang ingin dicari
2. Program melakukan pencarian dari data pertama hingga terakhir
3. Jika ditemukan, data akan ditampilkan
4. Jika tidak ditemukan, program akan menampilkan pesan bahwa data tidak ada

Kelebihan:

- Tidak memerlukan data terurut

Kekurangan:

- Kurang efisien untuk jumlah data yang besar

4.3.5 Fungsi Binary Search

Fungsi:
binarySearchUnit()
Digunakan untuk mencari data berdasarkan jumlah unit rumah.
Tahapan proses:

1. Data disalin ke array sementara
2. Data diurutkan terlebih dahulu menggunakan insertion sort
3. Pencarian dilakukan dengan metode pembagian dua (binary search)

Cara kerja:

- Menentukan posisi tengah
- Membandingkan nilai tengah dengan nilai yang dicari
- Mempersempit area pencarian hingga data ditemukan atau tidak ditemukan

Kelebihan:

- Lebih cepat dibandingkan sequential search

Kekurangan:

- Harus menggunakan data yang sudah terurut

4.3.6 Fungsi Login

Fungsi:
login()
Digunakan untuk membatasi akses ke program.
Cara kerja:

- Pengguna memasukkan username dan password
- Password disembunyikan menggunakan karakter *
- Pengguna diberikan maksimal 3 kali percobaan

Jika login berhasil, program akan dilanjutkan ke menu utama. Jika gagal, program akan berhenti.

4.3.7 Fungsi Tambah Data

Fungsi:
tambahData()
Digunakan untuk menambahkan data developer baru ke dalam array.
Program akan:

- Meminta input data dari pengguna
- Menyimpan data ke array
- Menambah jumlah data

4.3.8 Fungsi Tampil Data

Fungsi:
tampilData()
Digunakan untuk menampilkan seluruh data developer dalam bentuk tabel.
Jika tidak ada data, program akan menampilkan pesan bahwa data masih kosong.

4.3.9 Fungsi Ubah Data

Fungsi:
ubahData()
Digunakan untuk mengubah data yang sudah ada.
Pengguna akan:

- Memilih nomor data
- Menginput data baru
- Data lama akan digantikan dengan data baru

4.3.10 Fungsi Hapus Data

Fungsi:
hapusData()
Digunakan untuk menghapus data berdasarkan nomor.
Setelah data dihapus:

- Data setelahnya akan digeser ke depan
- Jumlah data akan berkurang

4.4 Fungsi Main (Menu Utama)

Fungsi main() merupakan pusat kendali program.
Program dimulai dengan:

1. Inisialisasi username dan password
2. Proses login
3. Jika login berhasil, program menampilkan menu utama

Menu utama terdiri dari:

1. Tambah Data
2. Tampilkan Data
3. Ubah Data
4. Hapus Data
5. Sorting Nama
6. Sorting Harga
7. Sorting Jumlah Unit
8. Sequential Search
9. Binary Search
10. Keluar

Program menggunakan struktur perulangan do-while sehingga menu akan terus ditampilkan sampai pengguna memilih keluar.

4.5 Penjelasan Error Handling pada Program

Program ini menerapkan mekanisme error handling menggunakan konsep exception handling dalam C++. Tujuan dari penerapan ini adalah untuk menangani berbagai kondisi kesalahan secara terstruktur tanpa menghentikan eksekusi program secara tiba-tiba.
1. Konsep Dasar yang Digunakan

Penanganan kesalahan dilakukan :

- try → untuk membungkus kode yang berpotensi menghasilkan error
- throw → untuk melempar exception saat terjadi kondisi tidak valid
- catch → untuk menangkap dan menangani exception

Program ini juga memanfaatkan custom exception yang telah didefinisikan dalam modul validator.h, seperti:

- DataKosongException
- DataPenuhException
- InputTidakValidException
- NilaiNegatifException
- LoginGagalException

---

2. Error Handling pada Proses Input

Pada beberapa bagian program, terutama input data, digunakan fungsi dari namespace Validator seperti:

- bacaInteger()
- bacaLong()
- validasiRentang()
- validasiPositif()
- validasiStringTidakKosong()

Fungsi-fungsi tersebut akan melempar exception apabila input:

- tidak sesuai tipe data
- berada di luar batas yang ditentukan
- bernilai kosong atau negatif

Exception yang dilempar kemudian ditangkap menggunakan blok catch, dan ditampilkan melalui fungsi cetakError().

---

3. Error Handling pada Operasi CRUD

a. Tambah Data

Pada fungsi tambahData, digunakan:

- DataPenuhException jika kapasitas penuh
- InputTidakValidException jika input tidak sesuai
- NilaiNegatifException jika nilai numerik tidak valid

Semua exception ditangkap dan ditampilkan tanpa menghentikan program.

---

b. Tampil Data

Pada fungsi tampilData:

- Jika tidak ada data, maka dilempar DataKosongException
- Exception ditangkap dan ditampilkan kepada pengguna

---

c. Ubah Data

Pada fungsi ubahData:

- DataKosongException jika data kosong
- InputTidakValidException jika input salah

- NilaiNegatifException untuk validasi angka

---

d. Hapus Data
Pada fungsi hapusData:
- DataKosongException jika tidak ada data
- InputTidakValidException jika nomor data tidak valid

---

4. Error Handling pada Proses Pencarian
a. Sequential Search
- Menggunakan DataKosongException jika data kosong
- Menggunakan InputTidakValidException untuk validasi input string

b. Binary Search
- DataKosongException jika data kosong
- InputTidakValidException jika input bukan angka
- NilaiNegatifException jika nilai tidak valid

---

5. Error Handling pada Login
Pada fungsi login:
- Sistem memberikan maksimal 3 kali percobaan login
- Jika gagal, maka dilempar LoginGagalException
- Exception ditangkap dan program menghentikan akses pengguna

---

6. Error Handling pada Menu Utama
Pada bagian menu utama:
```
try {
  pilihanMenu = Validator::bacaInteger("Pilihan menu");
  Validator::validasiRentang(pilihanMenu, 1, 10, "Pilihan menu");
}
```
Jika terjadi kesalahan:
- InputTidakValidException akan ditangkap
- Program menampilkan pesan error
- Menggunakan continue untuk mengulang input menu

---

7. Mekanisme Output Error
Semua pesan kesalahan ditampilkan menggunakan fungsi:
```
cetakError(e.what());
```
Fungsi ini:
- Menampilkan pesan error dengan warna merah
- Membantu membedakan antara output normal dan kesalahan

4.6 Fungsi Program Validator (validator.h)

Program validator.h merupakan modul yang digunakan untuk melakukan validasi terhadap data masukan (input) serta menangani kesalahan menggunakan mekanisme exception handling dalam bahasa C++. Modul ini bertujuan untuk memastikan bahwa setiap input yang diterima oleh program sesuai dengan tipe dan aturan yang telah ditentukan.
Modul ini diawali dengan penggunaan header guard (#ifndef, #define, dan #endif) yang berfungsi untuk mencegah terjadinya penyertaan file secara berulang (multiple inclusion) yang dapat menyebabkan kesalahan saat proses kompilasi.
Selanjutnya, program mengimpor beberapa pustaka standar yaitu <iostream>, <string>, <stdexcept>, dan <limits>. Pustaka tersebut digunakan untuk mendukung proses input-output, pengolahan string, pembuatan exception, serta pengelolaan buffer input.
Pada modul ini didefinisikan beberapa kelas exception turunan dari std::exception, yaitu DataPenuhException, DataKosongException, IndexTidakValidException, LoginGagalException, InputTidakValidException, dan NilaiNegatifException. Setiap kelas memiliki fungsi what() yang mengembalikan pesan kesalahan spesifik sesuai dengan kondisi yang terjadi. Penggunaan custom exception ini bertujuan agar penanganan kesalahan menjadi lebih terstruktur dan informatif.
Selain itu, terdapat namespace Validator yang berisi sekumpulan fungsi untuk membaca dan memvalidasi input dari pengguna. Fungsi bacaInteger dan bacaLong digunakan untuk membaca input numerik dari pengguna. Apabila input yang diberikan tidak sesuai dengan tipe data yang diharapkan, maka fungsi akan melempar exception InputTidakValidException. Kedua fungsi ini juga melakukan pembersihan buffer input untuk mencegah kesalahan pembacaan data berikutnya.
Fungsi validasiRentang digunakan untuk memastikan bahwa nilai yang dimasukkan berada dalam batas minimum dan maksimum tertentu. Jika nilai berada di luar rentang tersebut, maka akan dilempar exception. Fungsi validasiStringTidakKosong digunakan untuk memastikan bahwa input berupa string tidak kosong. Sementara itu, fungsi validasiPositif dan validasiPositifLong digunakan untuk memvalidasi bahwa nilai numerik yang dimasukkan lebih besar dari nol, dan akan melempar exception jika nilai tersebut nol atau negatif.
Secara keseluruhan, modul validator.h bekerja dengan cara memeriksa setiap input yang diterima, kemudian melempar exception apabila terjadi kesalahan. Exception tersebut selanjutnya ditangani pada program utama menggunakan blok try-catch, sehingga program dapat memberikan pesan kesalahan kepada pengguna tanpa menghentikan eksekusi secara tiba-tiba.

4. OUTPUT PROGRAM

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ LOGIN SISTEM - PERCOBAAN 1/3 ]
------------------------------------------------------------------------------
  Username (Nama) : Heiza
  Password (NIM)  : ***

  Login berhasil! Selamat datang, Heiza!
Press any key to continue . . .
```

Login

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  MENU UTAMA
------------------------------------------------------------------------------
  [1]  Tambah Data
  [2]  Tampilkan Data
  [3]  Ubah Data
  [4]  Hapus Data
------------------------------------------------------------------------------
  -- SORTING --
  [5]  Sort Nama Developer (Bubble Sort  - Ascending)
  [6]  Sort Harga Rumah    (Selection Sort - Descending)
  [7]  Sort Jumlah Unit    (Insertion Sort - Ascending)
------------------------------------------------------------------------------
  -- PENCARIAN --
  [8]  Cari Nama Developer  (Sequential Search)
  [9]  Cari Jumlah Unit     (Binary Search)
------------------------------------------------------------------------------
  [10] Keluar
==============================================================================
  Pilih Menu :
```

Menu Utama

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ TAMBAH DATA DEVELOPER PERUMAHAN ]
------------------------------------------------------------------------------
  Nama Developer    : Mumbo
  Nama Perumahan    : Lepard
  Lokasi Perumahan  : 950
  Jumlah Unit Rumah: 750
  Harga Rumah (Rp)  : 900000000

  [OK] Data berhasil ditambahkan!
------------------------------------------------------------------------------
Press any key to continue . . .
```

Create Data Perumahan

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ DAFTAR DATA DEVELOPER PERUMAHAN ]
------------------------------------------------------------------------------
No  Nama Developer    Nama Perumahan      Lokasi         Unit    Harga (Rp)
------------------------------------------------------------------------------
1   Lucius            Lumining            Jl. SetiaBudi  180     500000000
2   Merito            Melevol             Jl. Anggrek Mekar170     165000000
3   Lovinian          Wallnut             Jl. Dipenogoro 675     670000000
4   Mumbo             Lepard              950            750     900000000
------------------------------------------------------------------------------
  Total data (rekursif) : 4 entri
------------------------------------------------------------------------------
Press any key to continue . . .
```

Read Data Perumahan

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ UBAH DATA DEVELOPER ]
------------------------------------------------------------------------------
  Masukkan nomor data (1-4) : 4
  Nama Developer Baru  : Toronto Dev
  Nama Perumahan Baru  : Melovent
  Lokasi Baru          : Jl. Kancil
  Jumlah Unit Baru     : 860
  Harga Baru (Rp)      : 760000000

  [OK] Data berhasil diubah!
------------------------------------------------------------------------------
Press any key to continue . . .
```

Update Data Perumahan

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ HAPUS DATA DEVELOPER ]
------------------------------------------------------------------------------
  Masukkan nomor data (1-4) : 4

  [OK] Data nomor 4 berhasil dihapus!
------------------------------------------------------------------------------
Press any key to continue . . .
```

Delete Data Perumahan

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  MENU UTAMA
------------------------------------------------------------------------------
  [1]  Tambah Data
  [2]  Tampilkan Data
  [3]  Ubah Data
  [4]  Hapus Data
------------------------------------------------------------------------------
  -- SORTING --
  [5]  Sort Nama Developer (Bubble Sort  - Ascending)
  [6]  Sort Harga Rumah    (Selection Sort - Descending)
  [7]  Sort Jumlah Unit    (Insertion Sort - Ascending)
------------------------------------------------------------------------------
  -- PENCARIAN --
  [8]  Cari Nama Developer  (Sequential Search)
  [9]  Cari Jumlah Unit     (Binary Search)
------------------------------------------------------------------------------
  [10] Keluar
==============================================================================
  Pilih Menu : 5

  [OK] Data berhasil diurutkan berdasarkan Nama (Ascending).
Press any key to continue . . .
```

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ DAFTAR DATA DEVELOPER PERUMAHAN ]
------------------------------------------------------------------------------
No  Nama Developer    Nama Perumahan      Lokasi         Unit    Harga (Rp)
------------------------------------------------------------------------------
1   Lovinian          Wallnut             Jl. Dipenogoro 675     670000000
2   Menerva           Mempish             Jl. Soetmrin   950     900000000
3   Merito            Melevol             Jl. Anggrek Mekar170     165000000
------------------------------------------------------------------------------
  Total data (rekursif) : 3 entri
------------------------------------------------------------------------------
Press any key to continue . . .
```

Sorting Nama Perumahan

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  MENU UTAMA
------------------------------------------------------------------------------
  [1]  Tambah Data
  [2]  Tampilkan Data
  [3]  Ubah Data
  [4]  Hapus Data
------------------------------------------------------------------------------
  -- SORTING --
  [5]  Sort Nama Developer (Bubble Sort  - Ascending)
  [6]  Sort Harga Rumah    (Selection Sort - Descending)
  [7]  Sort Jumlah Unit    (Insertion Sort - Ascending)
------------------------------------------------------------------------------
  -- PENCARIAN --
  [8]  Cari Nama Developer  (Sequential Search)
  [9]  Cari Jumlah Unit     (Binary Search)
------------------------------------------------------------------------------
  [10] Keluar
==============================================================================
  Pilih Menu : 6

  [OK] Data berhasil diurutkan berdasarkan Harga (Descending).
Press any key to continue . . .
```

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ DAFTAR DATA DEVELOPER PERUMAHAN ]
------------------------------------------------------------------------------
No  Nama Developer    Nama Perumahan      Lokasi         Unit    Harga (Rp)
------------------------------------------------------------------------------
1   Menerva           Mempish             Jl. Soetmrin   950     900000000
2   Lovinian          Wallnut             Jl. Dipenogoro 675     670000000
3   Merito            Melevol             Jl. Anggrek Mekar170     165000000
------------------------------------------------------------------------------
  Total data (rekursif) : 3 entri
------------------------------------------------------------------------------
Press any key to continue . . .
```

Sorting Harga Perumahan

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  MENU UTAMA
------------------------------------------------------------------------------
  [1]  Tambah Data
  [2]  Tampilkan Data
  [3]  Ubah Data
  [4]  Hapus Data
------------------------------------------------------------------------------
  -- SORTING --
  [5]  Sort Nama Developer (Bubble Sort  - Ascending)
  [6]  Sort Harga Rumah    (Selection Sort - Descending)
  [7]  Sort Jumlah Unit    (Insertion Sort - Ascending)
------------------------------------------------------------------------------
  -- PENCARIAN --
  [8]  Cari Nama Developer  (Sequential Search)
  [9]  Cari Jumlah Unit     (Binary Search)
------------------------------------------------------------------------------
  [10] Keluar
==============================================================================
  Pilih Menu : 7

  [OK] Data berhasil diurutkan berdasarkan Jumlah Unit (Ascending).
Press any key to continue . . .
```

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ DAFTAR DATA DEVELOPER PERUMAHAN ]
------------------------------------------------------------------------------
No  Nama Developer    Nama Perumahan      Lokasi         Unit    Harga (Rp)
------------------------------------------------------------------------------
1   Merito            Melevol             Jl. Anggrek Mekar170     165000000
2   Lovinian          Wallnut             Jl. Dipenogoro 675      670000000
3   Menerva           Mempish             Jl. Soetmrin   950      900000000
------------------------------------------------------------------------------
  Total data (rekursif) : 3 entri
------------------------------------------------------------------------------
Press any key to continue . . .
```

Sorting Jumlah Unit

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ CARI DATA - SEQUENTIAL SEARCH (NAMA DEVELOPER) ]
------------------------------------------------------------------------------
  Masukkan nama developer yang dicari : Menerva
------------------------------------------------------------------------------
  Data ditemukan pada nomor 3
------------------------------------------------------------------------------
  Nama Developer  : Menerva
  Nama Perumahan  : Mempish
  Lokasi          : Jl. Soetmrin
  Jumlah Unit     : 950 unit
  Harga Rumah     : Rp 900000000
------------------------------------------------------------------------------
Press any key to continue . . .
```

Cari Nama Developer

```
==============================================================================
  SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN
==============================================================================
  [ CARI DATA - BINARY SEARCH (JUMLAH UNIT RUMAH) ]
------------------------------------------------------------------------------
  Masukkan jumlah unit yang dicari : 675
------------------------------------------------------------------------------
  Data ditemukan!
------------------------------------------------------------------------------
  Nama Developer  : Lovinian
  Nama Perumahan  : Wallnut
  Lokasi          : Jl. Dipenogoro
  Jumlah Unit     : 675 unit
  Harga Rumah     : Rp 670000000
------------------------------------------------------------------------------
Press any key to continue . . .
```

Cari Jumlah Unit Rumah

5. Langkah-langkah GIT

```
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git init
Initialized empty Git repository in C:/Users/Lucius/Documents/Praktikum-Semester-2/APL/.git/
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git add .
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git commit -m "First Upt ✅"
[master (root-commit) 2c92f6d] First Upt ✅
 2 files changed, 82 insertions(+)
 create mode 100644 Praktikum-APL/post-test/2509106019-HeizaRizki-PT-1.cpp
 create mode 100644 Praktikum-APL/post-test/2509106019-HeizaRizki-PT-1.exe
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git branch -M main
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git remote add origin https://github.com/01001100-KMS/APL.git
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 48.00 KiB | 2.00 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/01001100-KMS/APL.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL>
```

5.1 GIT Add

```
PS C:\Users\Lucius\Documents\New folder> git add .
```

Fungsinya**:** Memilih file yang sudah diubah untuk masuk ke staging **area** (daftar siap commit). Tanpa git add, perubahan tidak akan ikut tersimpan saat git commit.

Contoh:
git add index.html
→ hanya file index.html yang siap di-commit.
git add .
→ semua file yang berubah akan masuk staging area.

5.2 GIT Commit

```
PS C:\Users\Lucius\Documents\New folder> git commit -m "ini commit"
[main 94354ce] ini commit
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Fungsinya: Menyimpan perubahan yang sudah dipilih (staging area*) ke dalam riwayat repository Commit ini ibarat checkpoint atau simpan versi dari proyek. Biasanya commit disertai pesan (-m) agar jelas maksud perubahannya. Contoh:*

commit - git m "Menambahkan fitur login"

5.3 GIT Push

Fungsinya: Mengirim perubahan (commit) yang ada di repository lokal ke repository remote (misalnya GitHub, GitLab, Bitbucket). Supaya bisa push, biasanya harus sudah git remote add origin <url> terlebih dahulu.

Contoh:
```
git push origin main
```

→ Mengirim commit lokal ke branch main di repository remote bernama origin.